Dimensionality Reduction Approaches to Insider Threat Detection: From Principal Components to Interpretable Sparse Features

by

Suramya Raj Angdembay

A Thesis

Submitted to the Honors College of

The University of Southern Mississippi

in Partial Fulfillment

of Honors Requirements

Month Year

Approved by:

Dr. Haiyan Tian, Degree

School of Mathematics and Natural Sciences

Thesis Co-Advisor Name, Degree

School of Your School

Director Name, Degree

School of Your School

Joyce Inman, Ph.D., Dean

Honors College

ABSTRACT

Insider-threat detection requires distinguishing unusual activity from malicious activity. This thesis studies that distinction through principal component analysis, language-model surprisal, and sparse representations of a model's internal states on the synthetic CERT release 4.2 dataset. PCA scores daily activity by its reconstruction error. A language model trained on benign users scores the same kind of activity by how difficult its textual description is to predict. Separating predictions about a user's profile from predictions about their behavior improves the language model's ranking of insiders without retraining. Sparse-feature analysis then examines where selected internal features activate and how their removal changes predictions. These features contribute to behavior prediction, but the measured response does not establish an attack-specific mechanism. The main conclusion is that improving an anomaly score and explaining the model's internal computation are separate achievements: excluding profile predictions improves ranking, while profile information can still influence the behavior predictions that remain.

Keywords: Insider Threat; Dimensionality Reduction; Principal Component Analysis (PCA); Surprisal; Large Language Models; Mechanistic Interpretability; Sparse Autoencoders; One-Class Anomaly Detection

DEDICATION

Your dedication text here.

ACKNOWLEDGMENTS

Your acknowledgments begin here. Recognize advisors, faculty, funding agencies, and institutional support.

Contents

ABSTRACT **2**

DEDICATION **3**

ACKNOWLEDGMENTS **4**

LIST OF ABBREVIATIONS **10**

1 Introduction **11**

1.1 Example 1st Level Heading 11

Example 2nd Level Heading 11

2 Principal Component Analysis **12**

2.1 Principal Components: Motivation and Formulation 12

2.2 Inverse PCA and Reconstruction Error 14

2.3 A Worked Example and Its Geometry 15

2.4 Higher Dimensions and Anomaly Scoring 16

2.5 The CERT Dataset and Insider Cases 21

2.6 PCA Experiment: Preparation and Results 23

2.7 Summary 25

3 Language-Model Surprisal and Its Decomposition **27**

3.1 From Reconstruction to Prediction 27

3.2 How a Neural Network Predicts the Next Token 28

3.3 Learning Normal Activity Through Prediction 32
3.4 From Token Surprisal to an Anomaly Score 35
3.5 Experimental Design and Activity Records 37
3.6 Results and the Question of Internal Mechanism 41

4 Interpretable Dimensionality Reduction of the Fine-Tuned Model 43

4.1 Overview and Motivation 43
4.2 Hidden States and the Adaptation Difference 43
4.3 A Sparse Dictionary and a Worked Example 44
4.4 Choosing the Features to Test 45
4.5 Where Features Activate 46
4.6 Causal Test: Removing Features and Re-Scoring 47
4.7 Corrected Mechanistic Results 48
4.8 Limitations and Conclusions 49

A Supporting Mathematics for Language-Model Scoring 51

A.1 Why Additive Surprise Has a Logarithmic Form 51
A.2 Surprisal Written in Terms of Logits 52
A.3 Maximum Likelihood in a Frequency Model 53
A.4 Why Subtracting Class Means Gives a Different Score 54
A.5 User AUC and Equal-Insider Fold Averages 55

REFERENCES 57

List of Tables

2.1 Rank-1 reconstruction of the four observations. 15
2.2 The three insider scenarios of CERT release 4.2, following the answer key. . . 22
2.3 CERT feature groups and training variability 24
2.4 PCA results for all 70 insiders 25

3.1 An illustrative profile–behavior ranking change 37
3.2 User-disjoint splits of CERT release 4.2: no user belongs to more than one group. 39
3.3 Corrected language-model score decomposition 41

4.1 Sparse-feature activation location 47
4.2 Corrected feature-removal effects 48

List of Figures

2.1 Illustration of PCA in two dimensions: PCA rotates the coordinate system so that PC1 captures the largest variance while PC2 captures the smallest. Projecting onto PC1 yields a one-dimensional representation with minimal loss when PC2 contributes little variance. 16

2.2 Illustration of PCA in three dimensions: PC1, PC2, and PC3 form an orthonormal basis with decreasing explained variance. Dropping PC3 projects data onto a best-fit plane. 17

2.3 A high-dimensional PCA illustration (17 features): PCA reveals dominant axes of variation among observations (countries). Large separation along PC1 indicates an observation that differs substantially along the primary pattern in the data. 18

2.4 Synthetic two-dimensional dataset with principal component directions drawn from the center of the data. 19

2.5 Dataset represented in the PCA coordinate system where each observation is described by its PC1 and PC2 coordinates. 20

2.6 Values of the first principal component for all observations. 20

2.7 Values of the second principal component for all observations. 21

2.8 PCA ROC on all 70 insiders and 79 held-out benign users, using the 309 varying features. The marked threshold flags 35 insiders and 8 benign users. The curve is recomputed from the saved user scores of the original PCA run; no PCA refit is involved. 26

3.1 A small feedforward network 29

LIST OF ABBREVIATIONS

APA	American Psychological Association
AUC	Area Under the (Receiver Operating Characteristic) Curve
CERT	CERT Division of the Software Engineering Institute
LoRA	Low-Rank Adaptation
PCA	Principal Component Analysis
QLoRA	Quantized Low-Rank Adaptation
ROC	Receiver Operating Characteristic
SAE	Sparse Autoencoder
USM	The University of Southern Mississippi

Chapter I

Introduction

Begin the first chapter here. Indent the first line of each paragraph.

1.1 Example 1st Level Heading

Text for first-level heading.

Example 2nd Level Heading

Text for second-level heading.

Example 3rd Level Heading (In Paragraph)

Example 3rd Level Heading. Continue paragraph text immediately after the bold heading.

Chapter II

Principal Component Analysis

2.1 Principal Components: Motivation and Formulation

Principal Component Analysis (PCA) is a classical technique for identifying orthogonal directions along which the data exhibits the greatest variance. In applications such as behavioral cybersecurity logs, the raw feature space can be high-dimensional and strongly correlated, which complicates visualization, statistical modeling, and anomaly detection. PCA addresses these issues by constructing an orthonormal coordinate system whose axes (the *principal components*) are ordered by the amount of variance they explain. This chapter presents the mathematical formulation of PCA in the general p-dimensional setting, the inverse transformation that reconstructs data from the reduced space, and the reconstruction error that motivates PCA-based anomaly scoring, followed by low- and high-dimensional illustrations. Historically, the geometric origins of PCA trace back to Pearson, while the familiar algebraic eigenvalue formulation was developed by Hotelling [1, 2, 3].

PCA represents each observation as a numerical vector in $\mathbb{R}^p$ and approximates the data with a linear subspace. Its variance criterion summarizes dominant variation; it does not guarantee that high-variance directions are the most useful ones for detecting anomalies. Euclidean distance must also be meaningful in the chosen feature space. Because PCA is sensitive to the relative magnitude of features, the features are standardized first.

Let $X \in \mathbb{R}^{n \times p}$ be a data matrix with n observations and p features, whose ith row is the observation $\mathbf{x}_i^\top$. With the sample mean $\bar{\mathbf{x}} = \frac{1}{n}\sum_{i=1}^{n} \mathbf{x}_i$ and the vector of ones $\mathbf{1} \in \mathbb{R}^n$, the centered data matrix is

$$X_c \;=\; X - \mathbf{1}\bar{\mathbf{x}}^\top,$$

which subtracts the mean from every observation. The sample covariance matrix is

$$S \;=\; \frac{1}{n-1} X_c^\top X_c \in \mathbb{R}^{p \times p}, \tag{2.1}$$

which is symmetric and positive semidefinite and therefore has an orthonormal eigendecomposition.

PCA looks for the unit direction $\mathbf{a}$ along which the projected scores $z_i = \mathbf{a}^\top(\mathbf{x}_i - \bar{\mathbf{x}})$ vary most. Their variance is $\mathbf{a}^\top S \mathbf{a}$, so the first principal direction solves

$$\max_{\mathbf{a} \in \mathbb{R}^p} \; \mathbf{a}^\top S \mathbf{a} \quad \text{subject to} \quad \mathbf{a}^\top \mathbf{a} = 1. \tag{2.2}$$

With a Lagrange multiplier λ, setting the gradient of $\mathbf{a}^\top S \mathbf{a} - \lambda(\mathbf{a}^\top \mathbf{a} - 1)$ to zero gives

$$S\mathbf{a} = \lambda \mathbf{a}. \tag{2.3}$$

The principal directions are therefore eigenvectors of S, and each eigenvalue is the variance explained along its direction [2, 3]. Ordering the eigenvalues $\lambda_1 \geq \cdots \geq \lambda_p \geq 0$ with orthonormal eigenvectors $\mathbf{u}_1, \ldots, \mathbf{u}_p$ gives $S = U \Lambda U^\top$, with $U = [\mathbf{u}_1 \;\; \cdots \;\; \mathbf{u}_p]$ and $\Lambda = \operatorname{diag}(\lambda_1, \ldots, \lambda_p)$.

To reduce the dimension to $q < p$, keep the first q directions, $U_q = [\mathbf{u}_1 \;\; \cdots \;\; \mathbf{u}_q] \in \mathbb{R}^{p \times q}$. The reduced data, or *principal component scores*, are

$$Z \;=\; X_c U_q \in \mathbb{R}^{n \times q}, \tag{2.4}$$

whose ith row is $\mathbf{z}_i^\top = (U_q^\top \mathbf{x}_{c,i})^\top$. Since the total variance is $\mathrm{tr}(S) = \sum_{j=1}^{p} \lambda_j$, the proportion of variance explained by the first q components is

$$\mathrm{PVE}(q) \;=\; \frac{\sum_{j=1}^{q} \lambda_j}{\sum_{j=1}^{p} \lambda_j}, \tag{2.5}$$

and q is commonly chosen so that $\mathrm{PVE}(q)$ exceeds a fixed threshold such as 0.95.

2.2 Inverse PCA and Reconstruction Error

PCA provides dimensionality reduction, and the *inverse PCA transformation* approximates the original data from the reduced space:

$$\widehat{X} \;=\; Z\,U_q^\top + \mathbf{1}\bar{\mathbf{x}}^\top, \qquad \widehat{\mathbf{x}}_i \;=\; U_q \mathbf{z}_i + \bar{\mathbf{x}}, \tag{2.6}$$

where Z is the reduced data, U_q holds the principal components and $\bar{\mathbf{x}}$ is the mean vector. For the centered observation $\mathbf{x}_{c,i} = \mathbf{x}_i - \bar{\mathbf{x}}$ this is $\widehat{\mathbf{x}}_{c,i} = U_q U_q^\top \mathbf{x}_{c,i}$, the orthogonal projection onto the principal subspace $\mathrm{span}\{\mathbf{u}_1, \ldots, \mathbf{u}_q\}$.

The reconstruction error measures the difference between the original and the reconstructed data. For one observation it is

$$e_i \;=\; \|\mathbf{x}_i - \widehat{\mathbf{x}}_i\|_2^2 \;=\; \|(I - U_q U_q^\top)\,\mathbf{x}_{c,i}\|_2^2, \tag{2.7}$$

the squared Euclidean distance from $\mathbf{x}_{c,i}$ to the principal subspace, and over n observations it is the mean squared error

$$\mathrm{MSE} \;=\; \frac{1}{n} \sum_{i=1}^{n} \|\mathbf{x}_i - \widehat{\mathbf{x}}_i\|_2^2. \tag{2.8}$$

A model built from normal data reconstructs normal observations well, so a large reconstruction error may indicate an anomaly. This is the basis of the PCA anomaly score used in this thesis.

2.3 A Worked Example and Its Geometry

For an illustration in the original units, consider four observations of two features,

$$X = \begin{bmatrix} 2 & 1 \\ 3 & 2 \\ 4 & 2 \\ 5 & 4 \end{bmatrix}, \qquad \bar{\mathbf{x}} = \begin{bmatrix} 3.5 \\ 2.25 \end{bmatrix}, \qquad S = \begin{bmatrix} 1.6667 & 1.5 \\ 1.5 & 1.5833 \end{bmatrix}.$$

The eigenvalues of S are $\lambda_1 \approx 3.125$ and $\lambda_2 \approx 0.125$, so the first principal component $\mathbf{u}_1 \approx (0.72,\ 0.69)$ explains about 96% of the variance. Keeping $q = 1$, the scores are $z \approx (-1.94,\ -0.53,\ 0.19,\ 2.28)$, and the inverse transformation $\widehat{\mathbf{x}}_i = z_i \mathbf{u}_1 + \bar{\mathbf{x}}$ gives the reconstructions and errors in Table 2.1.

Table 2.1: Rank-1 reconstruction of the four observations.

i	$\mathbf{x}_i$	$\widehat{\mathbf{x}}_i$	$e_i = \lVert \mathbf{x}_i - \widehat{\mathbf{x}}_i \rVert_2^2$
1	(2, 1)	(2.10, 0.91)	0.018
2	(3, 2)	(3.12, 1.88)	0.029
3	(4, 2)	(3.64, 2.38)	0.274
4	(5, 4)	(5.15, 3.82)	0.054

The third observation lies farthest from the line spanned by $\mathbf{u}_1$ and has the largest error, and $\text{MSE} = (0.018 + 0.029 + 0.274 + 0.054)/4 \approx 0.094$. The same operations apply unchanged in high dimensions.

In $\mathbb{R}^2$, PCA identifies a new pair of orthonormal axes (u_1, u_2) obtained by rotating the original coordinate system so that u_1 aligns with the direction of maximum spread. If the data are approximately distributed along a line, reducing from 2D to 1D projects the points onto the line spanned by u_1 and discards the orthogonal component (Figure 2.1). In $\mathbb{R}^3$, the

three directions (u_1, u_2, u_3) are ordered by $(\lambda_1, \lambda_2, \lambda_3)$, and reducing to 2D projects onto the plane spanned by (u_1, u_2) (Figure 2.2).

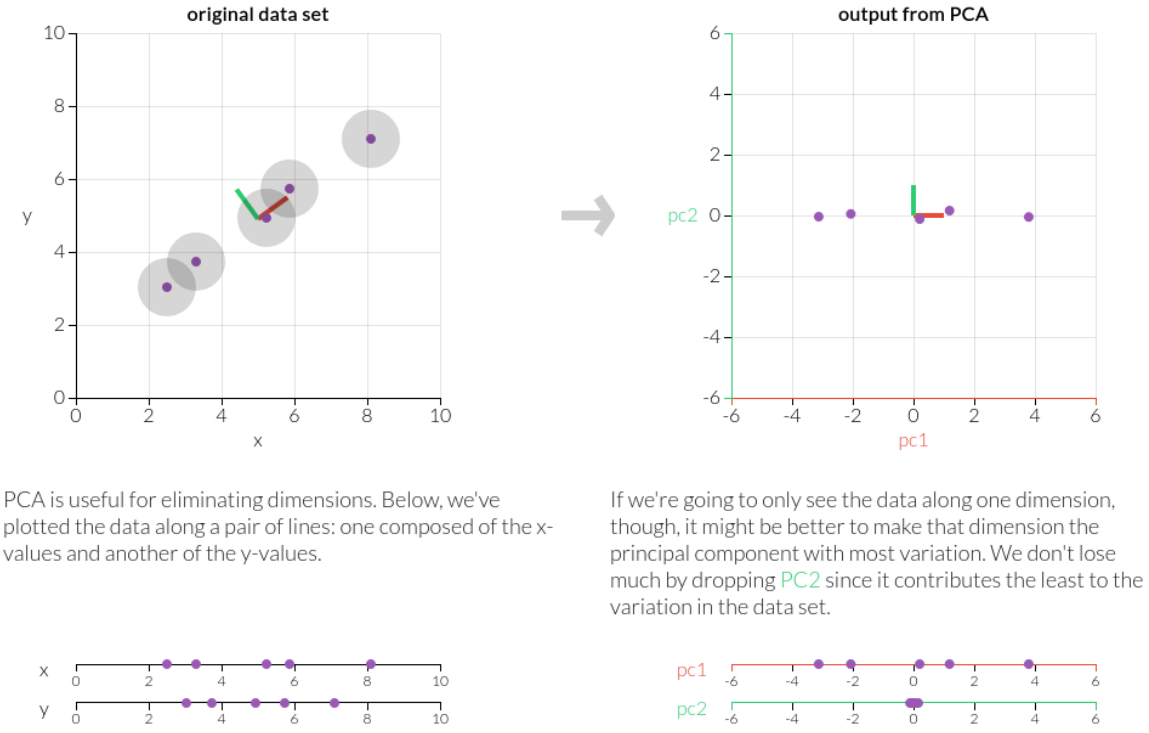


Figure 2.1: Illustration of PCA in two dimensions: PCA rotates the coordinate system so that PC1 captures the largest variance while PC2 captures the smallest. Projecting onto PC1 yields a one-dimensional representation with minimal loss when PC2 contributes little variance.

2.4 Higher Dimensions and Anomaly Scoring

PCA is often most valuable when p is large and direct visualization is not possible. To illustrate this, consider a dataset where each observation corresponds to a country and each feature corresponds to a measured quantity, producing points in $\mathbb{R}^{17}$. For example, one may treat the weekly per-person consumption of 17 food categories as a vector in $\mathbb{R}^{17}$. Although we cannot visualize $\mathbb{R}^{17}$ directly, PCA provides low-dimensional views (e.g., $q = 1$ or $q = 2$) that reveal dominant contrasts among countries.

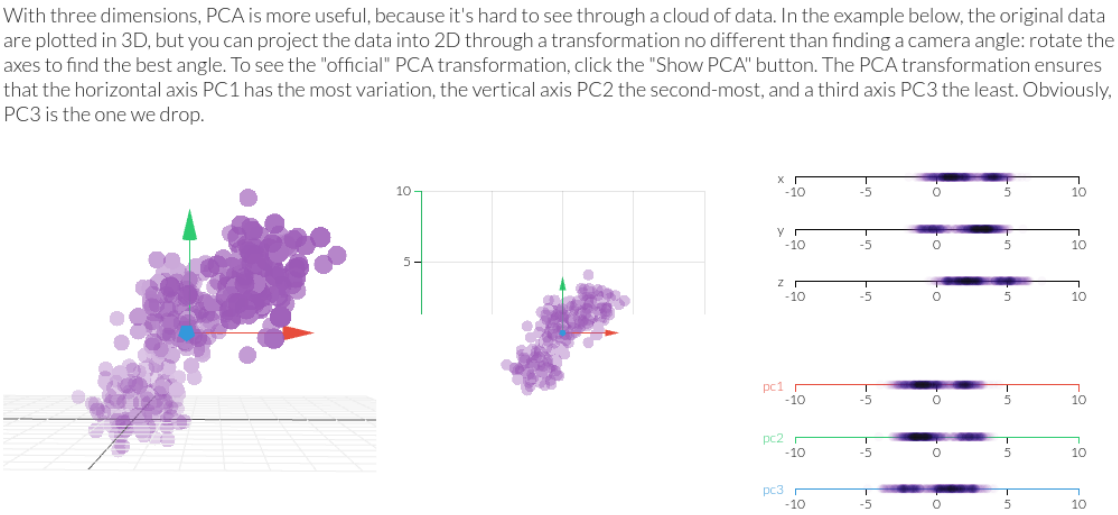


Figure 2.2: Illustration of PCA in three dimensions: PC1, PC2, and PC3 form an orthonormal basis with decreasing explained variance. Dropping PC3 projects data onto a best-fit plane.

Let $x_k \in \mathbb{R}^{17}$ denote the centered consumption vector for country k. The first PC score is $z_{k1} = u_1^\top x_k$, and the second is $z_{k2} = u_2^\top x_k$. Countries separated strongly along PC1 exhibit the greatest differences along the dominant consumption pattern, while PC2 captures the second most important pattern not explained by PC1.

A second illustration connects the geometry to anomaly scoring. To visualize how PCA organizes behavioral observations and highlights deviations from dominant patterns, we construct 30 two-dimensional points: 27 typical observations concentrated around a common trend, and three placed away from it to represent anomalous behavior. Figure 2.4 shows the data with the two principal directions drawn from its center; the first component aligns with the main trend and the second captures variation orthogonal to it. Projecting every observation onto these components gives the representation in Figure 2.5, and Figures 2.6 and 2.7 plot each observation's coordinate along the first and second components. Points that deviate from the general pattern become more clearly distinguishable in their component coordinates.

For a one-component reconstruction, an observation's error is the square of its discarded

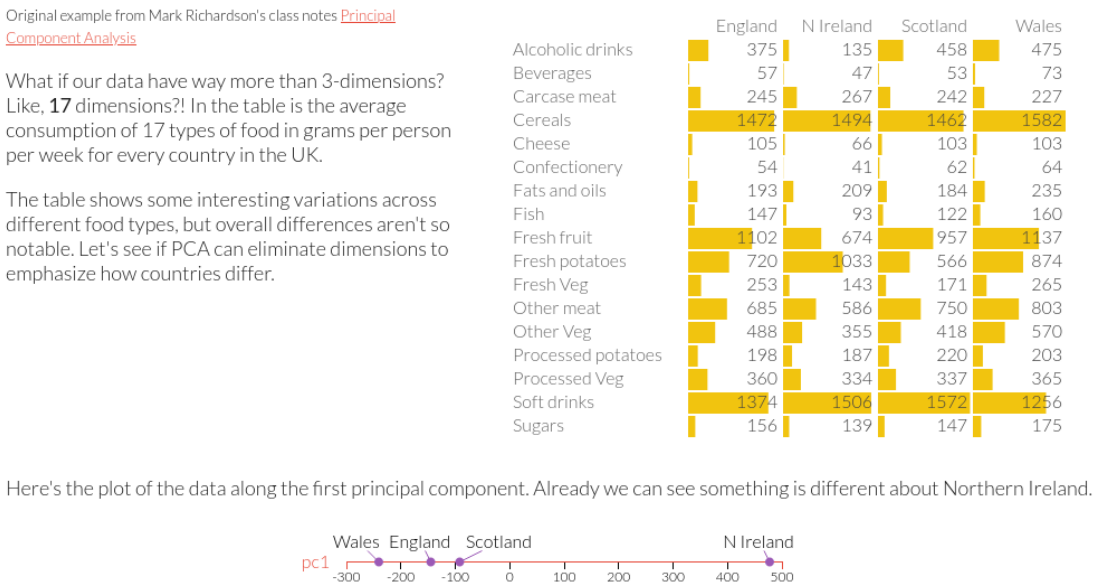


Figure 2.3: A high-dimensional PCA illustration (17 features): PCA reveals dominant axes of variation among observations (countries). Large separation along PC1 indicates an observation that differs substantially along the primary pattern in the data.

PC2 coordinate. A large PC1 coordinate alone is therefore not evidence of a large reconstruction error. These examples illustrate geometry; the CERT experiment below fits its components on benign training users only.

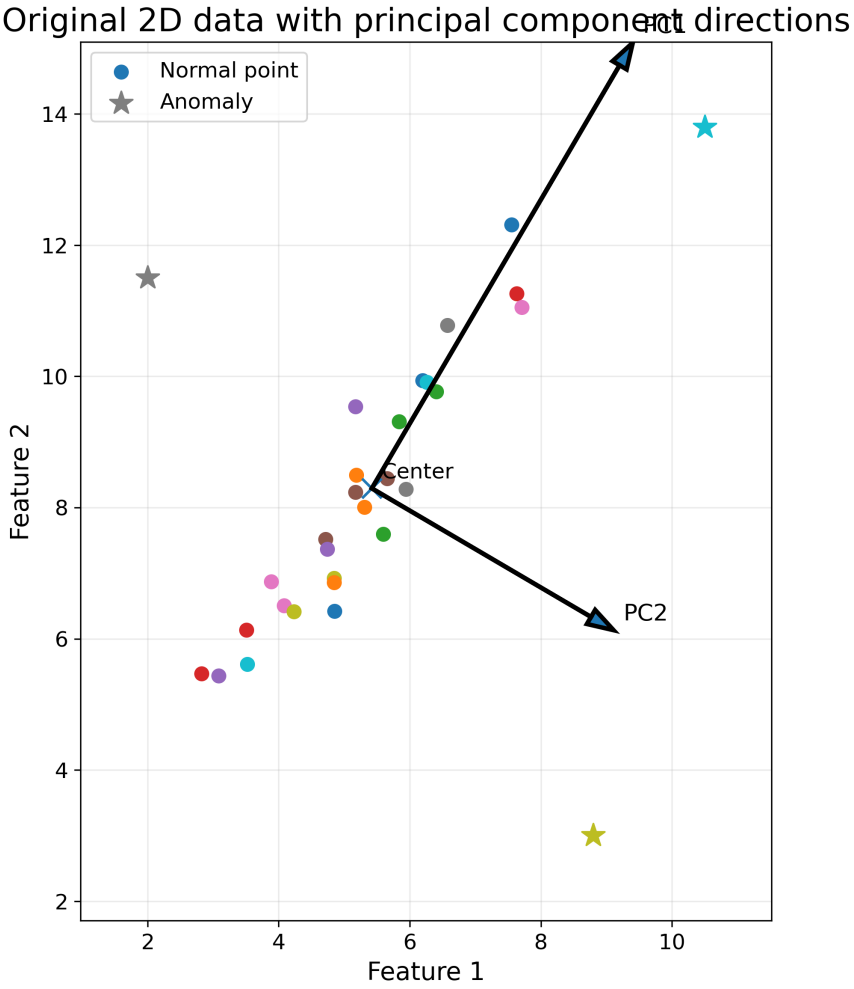


Figure 2.4: Synthetic two-dimensional dataset with principal component directions drawn from the center of the data.

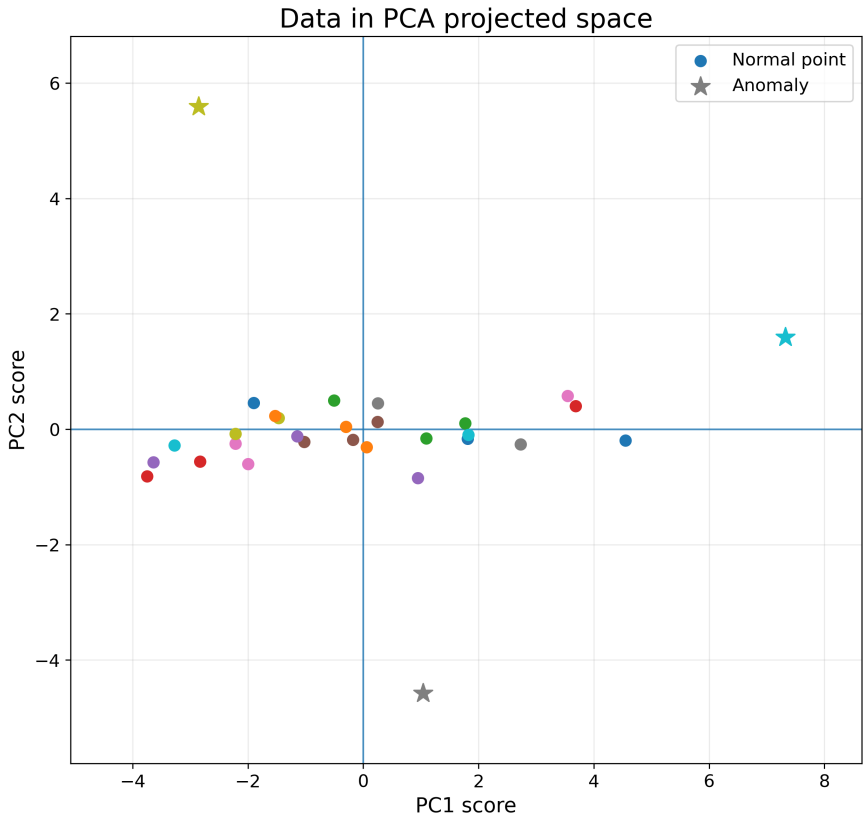


Figure 2.5: Dataset represented in the PCA coordinate system where each observation is described by its PC1 and PC2 coordinates.

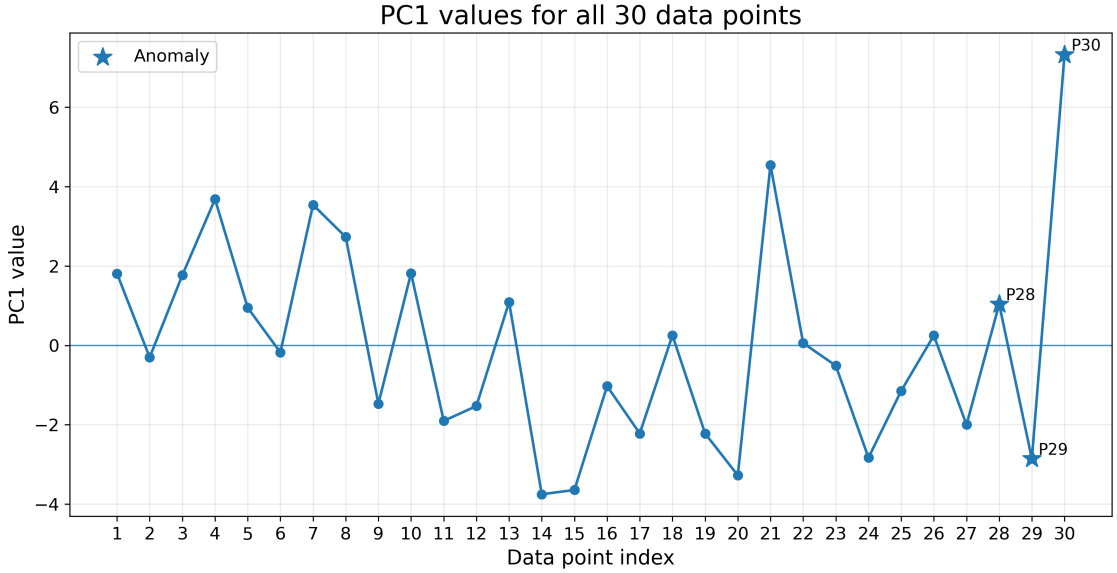


Figure 2.6: Values of the first principal component for all observations.

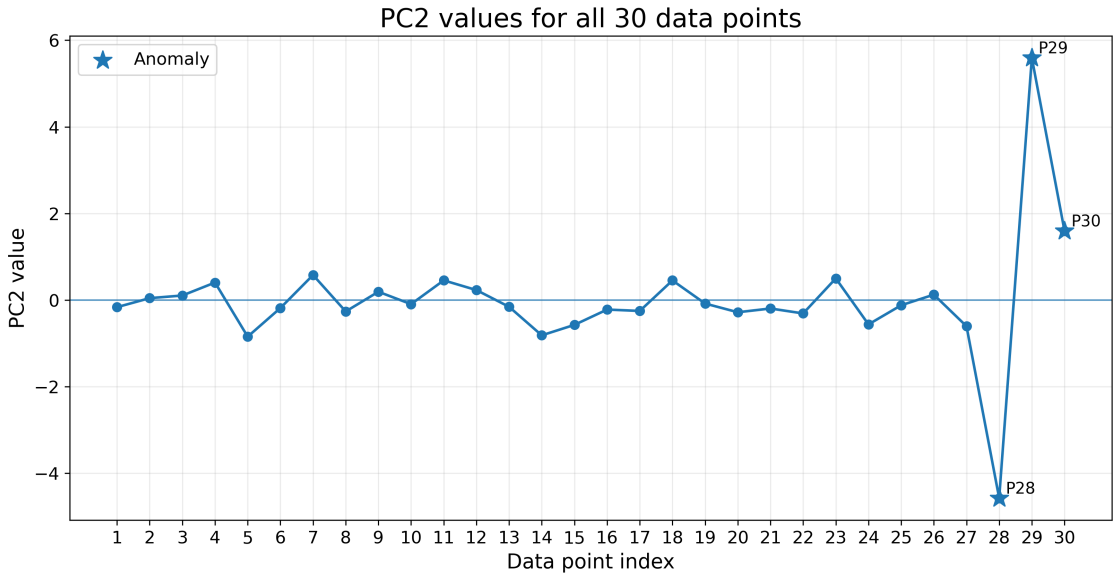


Figure 2.7: Values of the second principal component for all observations.

2.5 The CERT Dataset and Insider Cases

The CERT Insider Threat Test Dataset is a collection of synthetic datasets produced by the CERT Division of Carnegie Mellon University's Software Engineering Institute, with ExactData, LLC, under sponsorship from DARPA [5, 4]. Each dataset simulates an organization in which ordinary employees go about their work and a few simulated insiders act out a scripted malicious scenario. This thesis uses release 4.2, which simulates 1,000 employees whose recorded activity runs from 2 January 2010 to 17 May 2011. The data contain activity logs (logons and logoffs, removable-device connections, file operations, emails and web visits), organizational records (role, business unit, functional unit, department, team, and whether the employee is an IT administrator), and five personality scores for every employee.

Every log entry is an event $e = (u, \tau, s, \mathbf{a})$: a user $u \in \mathcal{U}$, a time τ, a source s (logon, device, file, email or web) and a list of attributes $\mathbf{a}$, with $|\mathcal{U}| = 1{,}000$. The unit of observation is the *user-day* $D = (u, t)$, one user on one calendar day, whose events are $E_{u,t} = \{e : e \text{ is an event of } u \text{ on day } t\}$. Every method in this thesis maps a user-day to an anomaly score $s(D) \in \mathbb{R}$, and a larger score means a more unusual day.

The dataset's answer key names each simulated insider, the scenario that insider follows, and the malicious events involved [4]. Release 4.2 contains 70 insiders in three scenarios (Table 2.2).

Table 2.2: The three insider scenarios of CERT release 4.2, following the answer key.

scenario	what the insider does	insiders
1	Starts working after hours and using a removable drive, which the user had not done before; uploads data to wikileaks.org and soon leaves.	30
2	Browses job websites and seeks work at a competitor; before leaving, copies data with a thumb drive at a much higher rate than before.	30
3	A disgruntled system administrator installs a keylogger on the supervisor's computer, logs in as the supervisor with the captured credentials and sends an alarming mass email, then leaves.	10

The language-model experiment uses the official incident window of each insider. If I_t is the calendar interval of day t, and $[a_u, b_u)$ is the answer key's incident interval, then

$$y(u,t) = \mathbb{I}[I_t \cap [a_u, b_u) \neq \varnothing] \tag{2.9}$$

for an official insider, and $y(u,t) = 0$ for other users. This identifies an *incident-window day*; it does not label every action on that day as malicious. An insider's other days are called *window-negative*, because absence from the window does not verify that all activity was benign. The calendar join uses December 28, 2009 as the day-index origin. Correct alignment gives 1,362 incident-window days from all 70 insiders among the 330,295 days with extracted sessions; another 530 calendar-window days have no session and are outside

that observation domain.

The PCA experiment uses a separate day-level feature table and its event-day flag: 966 days contain recorded malicious events (85, 861 and 20 by scenario). This is a different target from incident-window membership. Both experiments identify the same 70 official insiders, but their daily records and inputs differ, so their day-level metrics are not interchangeable.

Incident-window days account for about 0.41% of all session user-days. Calling every day negative would therefore give high accuracy while detecting no incidents. We instead evaluate rankings. A threshold on an anomaly score determines a true positive rate, the fraction of positive cases flagged, and a false positive rate, the fraction of negative cases flagged. The receiver operating characteristic (ROC) curve plots these rates as the threshold varies. Its area (AUC) is the probability that a positive case outranks a negative case, giving half credit for a tie. User-level AUC compares official insiders with non-insiders after taking each user's maximum daily score. A value of 1 is perfect ranking and 0.5 is chance-level ranking. This does not by itself measure which day an incident occurred.

2.6 PCA Experiment: Preparation and Results

The method above needs each observation as a vector of p numerical features, so the activity of each observation is summarized into one row $\mathbf{x}_i^\top$ of a matrix $X \in \mathbb{R}^{n \times p}$. Here an observation is a user-day, described by the day-level features of Le et al. [6] (Table 2.3). Release 4.2 gives $n = 330{,}452$ user-days with 502 features once identifiers and time indices are removed. Because PCA is sensitive to scale, each column is standardized with the mean and standard deviation of the training rows. Standardization needs a positive standard deviation, so the 193 columns that never vary among the training days are removed, leaving $p = 309$ features, or 299 without the organizational and personality attributes, the PCA counterpart of the behavior-only score of Chapter III.

PCA is fit on the 287,961 days of the 851 training users of Section 3.5, the users that also

Table 2.3: The day-level features of CERT release 4.2 [6], and how many of them vary among the training days. Activity features are also split by working hours and after hours and by the kind of computer used (the user's own, a shared one, another user's or the supervisor's). Files are split into six types and websites into six kinds, among them job-search, cloud-storage, leak and hacking websites. The profile attribute that does not vary is the business unit, which takes a single value in this release.

group	what is measured	features	varying
all activity	events of any kind	15	14
logons	logons	15	14
removable drives	connections and their mean duration	18	14
files	accesses by type, with mean length, depth and word count	210	132
emails	recipients, attachments, size and text length	42	33
web	visits by kind of website, with mean URL and content length	189	90
calendar	weekday or weekend	2	2
profile	organizational attributes and five personality scores	11	10
total		502	309

train the language model; all of them are benign, so PCA sees no insider and no held-out benign user. It keeps the first q principal directions U_q, with q the smallest number whose proportion of variance explained (2.5) reaches 95%; this gives $q = 71$ directions, which explain 95.1% of the variance ($q = 63$ without the profile attributes). Each observation is reduced to its scores and mapped back to the feature space with the inverse PCA transformation (2.6), and its anomaly score is its reconstruction error e_i (2.7); the mean squared error (2.8) summarizes a set of observations. Benign behavior is reconstructed well by the dominant directions of benign variation, so a large e_i marks an observation that those directions cannot explain.

The median reconstruction error is 6.6 on the days of the 79 held-out benign users and 17.0 on the 966 malicious days, and 6.4% of the malicious days exceed the 99th percentile of the training errors, against 0.5% of the held-out benign days. Scoring each user by their largest error gives the user-level AUCs of Table 2.4 and the ROC curve of Figure 2.8. For

all 70 insiders, the user AUC is 0.777. Removing profile attributes changes it little. The methods use different representations and sets of observed days; this is not a controlled comparison of model architectures. The last PCA row shows why the constant columns are removed. They include visits to leak and hacking websites and logons to the supervisor's computer, which no training day contains and which occur, in the evaluation data, almost only on malicious days. Left in their raw units, as some software does for a column with zero standard deviation, they carry 99.7% of the malicious days' squared error.

Table 2.4: PCA user-level AUC for all 70 official insiders against 79 held-out benign users, using each user's largest daily reconstruction error. Intervals resample both insider and benign users, with 10,000 draws. All rows retain 95% of training variance.

PCA input	p	q	user AUC [95% interval]
Varying features	309	71	0.777 [0.701, 0.848]
Varying behavior features	299	63	0.791 [0.716, 0.860]
All features, constants in raw units	502	71	0.832 [0.764, 0.894]

2.7 Summary

PCA describes high-dimensional observations through a small set of orthogonal directions. The discarded component gives a reconstruction error that can serve as an anomaly score. The CERT example also shows that preprocessing matters: retaining training-constant columns in raw units can make a few rare event fields dominate that error. The next chapter considers a different model of normal activity and asks which parts of its score help distinguish insiders.

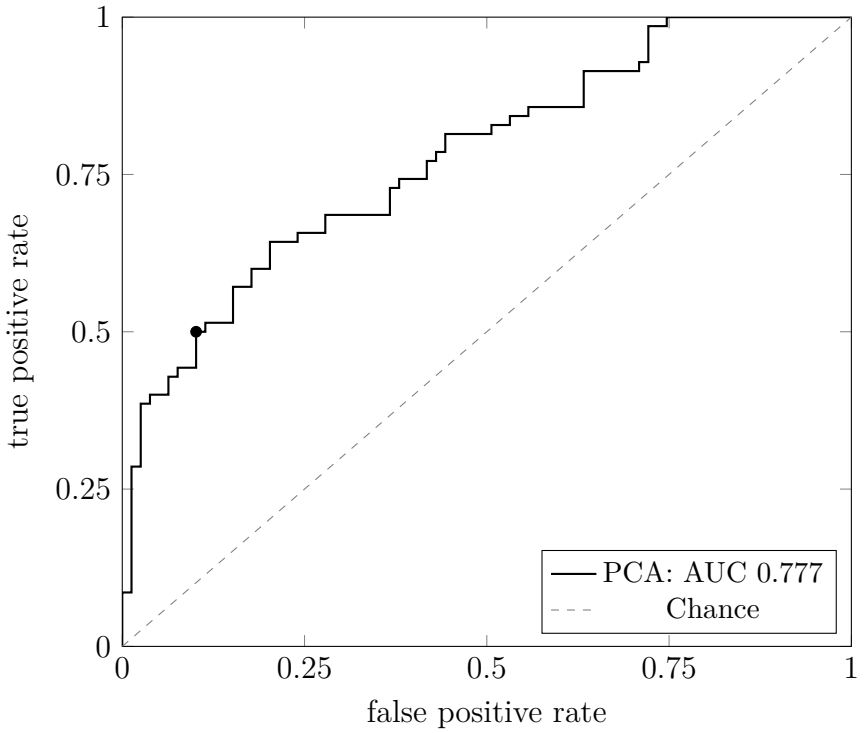


Figure 2.8: PCA ROC on all 70 insiders and 79 held-out benign users, using the 309 varying features. The marked threshold flags 35 insiders and 8 benign users. The curve is recomputed from the saved user scores of the original PCA run; no PCA refit is involved.

Chapter III

Language-Model Surprisal and Its Decomposition

3.1 From Reconstruction to Prediction

Chapter II scored an observation by the reconstruction error of a low-dimensional linear model. This chapter scores a user-day, one user's activity on one day, by how difficult its text record is for a large language model to predict. The language model is fine-tuned, or further trained, only on benign activity. Both scores ask the same question: how badly does a model of normal behavior account for this observation? Scoring activity by how poorly a model trained on normal activity predicts it follows earlier work on log and insider-threat data [10, 11].

To make the prediction task concrete, consider two fields from an activity record:

`dept=15`	the employee's department code,
`n_email=3`	the number of emails recorded for the session.

After reading the text up to `n_email=`, the model assigns probabilities to possible next pieces of text. We look up the probability it assigned to the piece actually observed, here the digit `3`. We repeat this prediction at successive positions in the record. Predictions that assign little probability to the observed text contribute more to the anomaly score.

This approach needs two ingredients: a network that assigns probabilities to text, and training that makes those probabilities reflect patterns in benign activity. We start with a pretrained language model, which has already learned to predict text from a large corpus.

We adapt it to benign employee records, then keep its learned weights fixed while scoring evaluation records. An unusual record receives a high score, but unusualness alone does not establish malicious intent.

The contribution of this chapter is to take that score apart. The record describes both *who* the employee is and *what* the employee did. We show that its average prediction loss splits exactly into contributions from those two kinds of text, and examine how counting those predictions changes the ranking of insiders. We first build the prediction and scoring machinery, then describe the experiment that tests this separation. The connection to dimensionality reduction returns when we restrict the changes made during training, and again in Chapter IV, where we examine changes in the network's internal representations.

3.2 How a Neural Network Predicts the Next Token

A neural network computes with numbers. Its basic operations combine input values using adjustable *weights*, then transform the result. Before considering a language model, take a single unit with two numerical inputs, named z_1 and z_2. Let w_1 and w_2 be their connection weights, and let b be an added offset, called a *bias*. The unit's output is called its *activation*, here h. For this teaching example, use the rectified linear unit (ReLU): keep a positive weighted sum and replace a negative one with zero. Writing max for the larger of its arguments gives

$$h = \max\{0,\ w_1 z_1 + w_2 z_2 + b\}. \tag{3.1}$$

For inputs $z_1 = 1$ and $z_2 = 2$, weights $w_1 = 2$ and $w_2 = -1$, and bias $b = 1$, the weighted sum is $2(1) - 1(2) + 1 = 1$, so $h = 1$. Keeping those weights fixed but changing the second input to $z_2 = 4$ makes the sum -1 and the activation zero. Thus the activation depends on the current input even when the weights do not change. These numbers illustrate a computation; they are not measurements from our model.

A layer performs this calculation with several units. To write it compactly, let d_{in} be

the number of input values and d_{out} the number of units. The column vector $\mathbf{z}$ contains the d_{in} inputs. The matrix W has d_{out} rows and d_{in} columns: each row contains one unit's connection weights. The vector $\mathbf{b}$ contains its d_{out} biases, and $\mathbf{h}$ contains the resulting activations. Finally, let ϕ mean applying ReLU to each entry. The entire layer is

$$\mathbf{h} = \phi(W\mathbf{z} + \mathbf{b}). \tag{3.2}$$

Matrix multiplication $W\mathbf{z}$ forms the weighted sum for every unit at once. A *multilayer perceptron* (MLP) passes the outputs of one layer into another. The nonlinear activation matters: without it, a succession of linear transformations and biases could be replaced by one transformation of the same kind. Figure 3.1 shows two inputs and three hidden activations, h_1, h_2, h_3, whose subscripts identify the units.

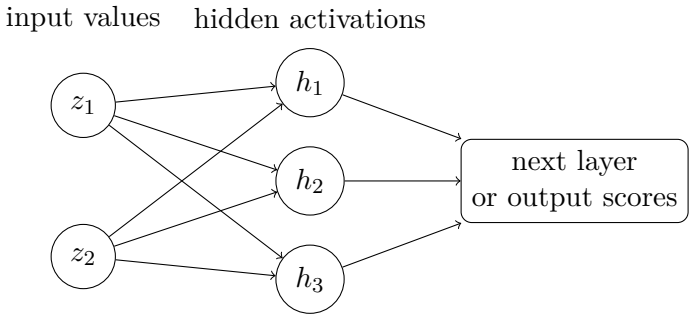


Figure 3.1: A small feedforward network. The two inputs z_1, z_2 feed three hidden units, whose activations are h_1, h_2, h_3; the subscripts identify the units. Each incoming arrow has a weight. Each hidden unit also has a bias, omitted from the drawing, and applies the operation in (3.1). The activations become inputs to the next calculation.

We use the symbol θ for the collection of stored numerical parameters that govern a network's computation. For the single unit above, this collection is $\theta = (w_1, w_2, b) = (2, -1, 1)$. The inputs z_1, z_2 and the computed activation h are not part of θ. For the layer in (3.2), the parameters are the entries of W and $\mathbf{b}$. For a complete language model, the collection also includes parameters in its input representations, attention operations, and output layer. Training changes the trainable members of this collection. By contrast, activations

are computed afresh from each input. This distinction is essential: a frozen model has fixed parameters, but different records still produce different internal activations.

To apply a network to text, a *tokenizer* divides the text into pieces called *tokens*. These may be words, parts of words, digits, or punctuation. Write the tokens of one record as $x_1, \ldots, x_n$, where n is the number of tokens and a subscript identifies a position. The symbol x_t denotes the token actually observed at position t; for a scored prediction, t is an integer from 2 to n. Its preceding text is abbreviated $x_{<t} = (x_1, \ldots, x_{t-1})$; the notation $< t$ means all positions before t. For example, if x_t is the digit `3` in `n_email=3`, its preceding tokens include the field name and equals sign.

The model first looks up a learned numerical vector for each token, called an *embedding*. These vectors enter a transformer network [12]. Attention combines information from the available token positions; MLP blocks transform the resulting vectors at each position. In next-token prediction, the attention pattern prevents a prediction from using the target token or later tokens. Position information, normalization of intermediate values, and connections that add a block's input to its output also form part of the computation.

Our model, Qwen3-8B, has 36 transformer layers and represents each position with 4,096 numbers [15, 16]. Its MLP blocks use a gated nonlinear operation, called SwiGLU, which multiplies one transformed branch by another. The small ReLU network above introduces the roles of weights and activations; it is not the exact formula for a Qwen block. Both the attention and MLP computations contribute to the representations used for prediction.

For the prediction at position t, call the final normalized vector at position $t-1$ $\mathbf{h}_{t-1}$, and let $d = 4096$ be its number of coordinates. The set $\mathcal{V}$ contains all candidate output tokens and is called the *vocabulary*. Write $V = |\mathcal{V}|$ for its size; vertical bars around a set mean the number of its members. A lowercase v denotes one candidate token, which need not equal the observed token x_t.

Each candidate v has a learned output-weight vector $\mathbf{w}_v$ with d coordinates. Its output score, or *logit*, is o_v. Let i count coordinates from 1 to d, so $w_{v,i}$ and $h_{t-1,i}$ are the corre-

sponding entries of the two vectors. The superscript $\top$ transposes a column into a row, and $\sum$ means to add the indicated terms. Their dot product is

$$o_v = \mathbf{w}_v^\top \mathbf{h}_{t-1} = \sum_{i=1}^{d} w_{v,i} h_{t-1,i}. \tag{3.3}$$

Collecting the row vectors into a matrix W_{out} with V rows and d columns gives all logits at once: $\mathbf{o} = W_{\text{out}}\mathbf{h}_{t-1}$, where $\mathbf{o}$ is the vector of V logits and the subscript *out* identifies the output layer.

Logits can be negative and need not add to one. The *softmax* operation converts them into probabilities. Let $p_\theta(x_t = v \mid x_{<t})$ mean the probability, under the model with parameters θ, of candidate v occurring at position t, given the preceding tokens. The vertical bar $\mid$ means *given*. In the denominator below, v' is a second candidate name used to sum over the entire vocabulary; its prime does not indicate a derivative. The symbol $\in$ means *belongs to*. Using e for the base of the natural exponential,

$$p_\theta(x_t = v \mid x_{<t}) = \frac{e^{o_v}}{\sum_{v' \in \mathcal{V}} e^{o_{v'}}}. \tag{3.4}$$

Each exponentiated logit is positive for finite logits, and dividing by their sum makes the probabilities add to one.

For a numerical illustration, let the earlier unit's activation $h = 1$ feed a toy output layer with just three candidate tokens: `0`, `3`, and `8`. Throughout this chapter, log and ln both mean the natural logarithm, the inverse of the exponential with base e. Suppose those candidates' output weights are 0, $\log 8$, and 0. Multiplying each weight by the activation gives logits 0, $\log 8$, and 0. Exponentiation gives 1, 8, and 1; their sum is 10. Softmax therefore gives probabilities 0.1, 0.8, and 0.1. If the observed token is `3`, we use its probability 0.8. If the input instead makes the activation zero, all three logits become zero and each candidate receives probability 1/3. This connects the earlier neural-network calculation to the prediction probabilities. The actual Qwen vocabulary is much larger; these are illustrative values,

not outputs measured in the experiment.

We can now follow the complete calculation: preceding text becomes token vectors, the network computes an internal representation, the output layer assigns candidate logits, and softmax produces probabilities. The next step is to measure how much probability the model gave to the observation that actually occurred.

3.3 Learning Normal Activity Through Prediction

We need a prediction penalty that is small when the observed token receives high probability and large when it receives low probability. For position t, call this penalty ℓ_t. Abbreviate the probability assigned to the actual observation by $p_\theta(x_t \mid x_{<t})$: this is the probability from (3.4) after choosing the candidate v to be the observed x_t. Its *surprisal* is

$$\ell_t = -\log p_\theta(x_t \mid x_{<t}). \tag{3.5}$$

The minus sign makes the value nonnegative because a probability is at most one. Using $\approx$ to mean approximately equal, the illustrative probability 0.8 gives $-\log(0.8) \approx 0.223$; probability 0.01 gives $-\log(0.01) \approx 4.605$. These values are measured in *nats*, the information unit obtained with a natural logarithm. A large surprisal means the model assigned little probability to what occurred.

Why use a logarithm? It turns a product of conditional probabilities into a sum of penalties. The probability assigned to a sequence is built from the probability of each successive observation given the preceding ones. For the record $x_1, \ldots, x_n$, we leave the first token unscored and condition on it. The probability $p_\theta(x_2, \ldots, x_n \mid x_1)$ is the model's probability of all remaining tokens in their observed order. The chain rule of probability gives

$$-\log p_\theta(x_2, \ldots, x_n \mid x_1) = \sum_{t=2}^{n} -\log p_\theta(x_t \mid x_{<t}) = \sum_{t=2}^{n} \ell_t. \tag{3.6}$$

Here the summation adds positions 2 through n. No independence between successive tokens is assumed: each probability depends on the preceding text. The resulting sum is the record's *negative log-likelihood*, a measure of how poorly the model predicts that observed record. Appendix A gives the additional justification for the logarithm and its relation to the logits.

This same penalty provides a training objective. Let $\mathcal{B}$ be the set of benign training days, and let D denote one day in that set. Its record has n_D tokens. Write $x_t^{(D)}$ for its token at position t and $x_{<t}^{(D)}$ for its preceding tokens; the superscript (D) identifies the record and is not a power. Let $J(\theta)$ be the total training loss for model parameters θ. Then

$$J(\theta) = \sum_{D \in \mathcal{B}} \sum_{t=2}^{n_D} -\log p_\theta\big(x_t^{(D)} \mid x_{<t}^{(D)}\big). \tag{3.7}$$

The inner sum adds prediction losses within one day; the outer sum adds across training days. Minimizing this loss is *maximum-likelihood training*: it seeks to increase the probability of the observed benign text. Training estimates how small changes to the trainable parameters affect the loss, then repeatedly updates them using gradient-based optimization. Neither a decrease at every individual update nor a globally best solution is guaranteed.

A simple frequency model helps explain the objective. If a token occurs 80 times among 100 observations, a model that ignores context minimizes its loss by assigning that token probability 0.8. Appendix A.3 derives this result. The language model learns a more flexible rule: its probabilities depend on the preceding record. Learning from benign activity motivates using low probability as an anomaly signal, but an unfamiliar legitimate activity can also be improbable.

We do not learn the whole network from scratch. Qwen3-8B already has parameters learned from a broad text corpus [15]. Our further training, called *fine-tuning* or *adaptation*, uses only benign employees' activity records. Thus benign-only describes our adaptation data, not all data used in the model's earlier development.

To limit the number of parameters that must be trained, we use *low-rank adaptation*

(LoRA) [13]. Consider one selected weight matrix W, with d_{in} input coordinates and d_{out} output coordinates, as in (3.2). Keep W fixed and add the product of two smaller trainable matrices. Let r be the chosen rank limit, G a matrix with d_{out} rows and r columns, and H a matrix with r rows and d_{in} columns. The effective matrix W' used after adaptation is

$$W' = W + GH, \qquad r = 16. \tag{3.8}$$

The prime identifies the adapted matrix. These added trainable factors are called adapters. A fixed LoRA scaling factor is absorbed into G in this notation. The formula describes evaluation, when dropout, the random masking of adapter inputs used during training, is disabled. The product GH can change the output only along combinations of the r columns of G. Its rank, the number of independent output directions, is therefore at most r. This is the connection to low-dimensional structure: each adapted matrix receives a restricted correction.

The two factors contain $r(d_{\text{in}} + d_{\text{out}})$ trainable entries instead of the $d_{\text{in}}d_{\text{out}}$ entries of the full matrix. For a 4,096-by-4,096 matrix, the rank-16 factors contain 131,072 entries, about 0.78% of the original 16,777,216. This restriction applies to each adapted matrix separately. Because the full network combines many matrices and nonlinear operations, its change in hidden activations need not lie in a 16-dimensional space.

The quantized version, QLoRA [14], stores the frozen base weights in a 4-bit representation to reduce memory use while learning these corrections. In (3.7), only the adapter parameters are allowed to change; the base parameters remain fixed. Once adaptation finishes, we also freeze the adapters. Training and evaluation then have distinct roles: training uses surprisal to update parameters, while evaluation uses surprisal to score records without updating them.

3.4 From Token Surprisal to an Anomaly Score

One token's surprisal measures one prediction. To compare entire days, we need to combine the predictions in each record. Fix a day D with n tokens, and let $\mathcal{T} = \{2, \ldots, n\}$ be its set of scored positions. There are $N = |\mathcal{T}| = n - 1$ such positions, since the first token has no preceding token in the record and is not scored. We use records with at least two tokens. Define the day's anomaly score $\bar{\ell}(D)$ as the average token surprisal; the bar indicates an average:

$$\bar{\ell}(D) = \frac{1}{N} \sum_{t \in \mathcal{T}} \ell_t. \tag{3.9}$$

This divides the sum of prediction penalties by the number of predictions. Averaging prevents a longer record from receiving a larger score simply because more terms were added. A large value means the adapted model found the day's text difficult to predict; it is not a probability that the user is malicious.

For user-level evaluation, let u identify an employee and let $\mathcal{D}_u$ be the set of that employee's observed days. We define the user score $\bar{\ell}(u)$ by the largest daily score:

$$\bar{\ell}(u) = \max_{D \in \mathcal{D}_u} \bar{\ell}(D). \tag{3.10}$$

The maximum selects the most unusual observed day. This is a choice of summary rule: users with longer histories have more opportunities to receive a high score. When we change which token predictions contribute to a daily score, we also recalculate each user's maximum using that score.

Now return to the two fields that introduced the chapter. Predicting `dept=15` concerns the profile; predicting `n_email=3` concerns activity. A rare department can be hard to predict even on an ordinary workday. Because both predictions contribute to the full score, unusual profile text could affect the ranking independently of unusual activity text.

In our records, lines headed `DAY` and `PSY` contain organizational and personality infor-

mation, while `SESSIONS` and `SES` give the session count and activity measurements. Call their two token groups profile (P) and behavior (B). This is an operational grouping by text position. The `DAY` line also includes a week field, so the profile group is not composed entirely of identity attributes. The full format appears in the next section.

Let $\mathcal{T}_P$ and $\mathcal{T}_B$ be the profile and behavior subsets of the scored positions. They do not overlap, and together they contain every scored position: $\mathcal{T} = \mathcal{T}_P \cup \mathcal{T}_B$, where $\cup$ means the union of two sets. Let $N_P = |\mathcal{T}_P|$ and $N_B = |\mathcal{T}_B|$ count their positions, so $N = N_P + N_B$. Define the summed losses L_P and L_B by

$$L_P = \sum_{t \in \mathcal{T}_P} \ell_t, \qquad L_B = \sum_{t \in \mathcal{T}_B} \ell_t.$$

Their sum $L = L_P + L_B$ is the full record's total loss. Both classes are nonempty in our scored records, so their average losses are $\bar{\ell}_P = L_P/N_P$ and $\bar{\ell}_B = L_B/N_B$. We temporarily omit the day argument (D) because all these quantities refer to this one record. Substituting the class totals into the full average gives

$$\bar{\ell} = \frac{L_P + L_B}{N} = \frac{N_P}{N}\bar{\ell}_P + \frac{N_B}{N}\bar{\ell}_B. \tag{3.11}$$

The fractions N_P/N and N_B/N are the classes' shares of scored tokens. The identity follows just from partitioning a sum; it makes no assumption that profile and behavior predictions are independent.

To score only behavior predictions, we remove the profile contribution from both the loss total and the token count:

$$\bar{\ell}_B = \frac{L_B}{N_B} = \frac{L - L_P}{N - N_P}. \tag{3.12}$$

The model and its input stay the same. A *target* is the token being predicted, and the target's group decides whether its loss is counted. Profile text still precedes the activity text and can influence its predictions. Thus behavior-only scoring changes the average, without

removing profile information from the model.

Consider two illustrative days with five scored tokens each: one profile token and four behavior tokens. Their profile and behavior shares are therefore $1/5 = 0.2$ and $4/5 = 0.8$. Day A belongs to a malicious user with a common profile and unusual behavior; Day B belongs to a benign user with a rare profile and routine behavior. Table 3.1 gives example class losses in nats. These values are chosen to explain the calculation, not drawn from the experiment.

Table 3.1: Class means and scores for two illustrative days. The full score weights profile loss by 0.2 and behavior loss by 0.8.

day	$\bar{\ell}_P$	$\bar{\ell}_B$	full $\bar{\ell}$	behavior only
A (malicious)	0.1054	0.9324	0.7670	0.9324
B (benign)	2.9957	0.4125	0.9291	0.4125

The full score places benign Day B above malicious Day A. Its weighted profile contribution favors B by about 0.578, outweighing a weighted behavior contribution that favors A by about 0.416. Counting only behavior predictions reverses that ordering. Whether this helps on the dataset is an empirical question, answered below. Simply subtracting the two class means is a different calculation; Appendix A.4 shows why it does not recover the behavior score.

3.5 Experimental Design and Activity Records

The method now specifies what we need from the experiment: activity records, benign users for adaptation, separate users for evaluation, and labels with which to assess the scores. We use CERT release 4.2, a synthetic dataset of ordinary employee activity and scripted insider incidents. Section 2.5 describes the incident scenarios and calendar-label rule.

The language model reads activity grouped into *sessions*, each running from a logon to its matching logoff on one computer. Features follow the extraction of Le et al. [6]: time of day, duration, computer, and counts of logon, device, file, email, and web events. We write these measurements as text to provide a consistent input to the pretrained model. It receives activity summaries, not employees' email message bodies.

Each day starts with a `DAY` line of organizational attributes and a `PSY` line of personality scores. A `SESSIONS` line gives the session count, followed by one `SES` line of measurements per session. The following is a complete benign record from the dataset, for user `AAF0535` on day 7. The session line is wrapped here to fit the page; in the actual input it is one line.

```
DAY week=1 project=na role=39 b_unit=0 f_unit=4 dept=15 team=11 itadmin=0
PSY O=17 C=21 E=36 A=33 N=31
SESSIONS total=1 kept=1
SES idx=0 pc=0 isworkhour=1 isafterhour=0 isweekend=0 isweekendafterhour=0
    duration=489 n_concurrent_sessions=1 start_with=1 end_with=1
    ses_start=8.933 ses_end=17.083 n_allact=34 n_logon=1 n_usb=0
    usb_mean_usb_dur=0 n_file=0 file_n-disk1=0 file_n_exef=0 n_email=3
    email_mean_n_atts=1.333 n_http=29 http_n_jobf=0 http_n_cloudf=0
    http_n_leakf=0 http_n_hackf=0
```

This employee had one working-hours session on their usual computer, lasting 489 minutes, with 3 emails and 29 web requests. The two fields used earlier are part of this record. The organizational fields are encoded categories; the letters `O`, `C`, `E`, `A`, and `N` denote openness, conscientiousness, extraversion, agreeableness, and neuroticism, the five personality scores.

For this representation and tokenizer, numbers are split into individual digits. For example, `dept=15` becomes `dept`, `=`, `1`, and `5`. The complete record has $N = 212$ scored positions, with $N_P = 54$ profile positions and $N_B = 158$ behavior positions. A position is assigned to the profile group if it falls on a `DAY` or `PSY` line; remaining positions, including session lines and residual text, belong to the behavior group. The digit `1` can therefore be a profile token in `team=11` and a behavior token in `n_logon=1`. The class of the target token determines

the class of its prediction loss.

The historical conversion program omitted four hyphenated file fields: `file_n-to_usb1`, `file_n-from_usb1`, `file_n-file_act3`, and `file_n-disk1`. If a missing field appears as a zero in the text, it is a default inserted during conversion, not a measured absence of activity. The findings concern this recorded representation.

Release 4.2 supplies 470,611 extracted sessions on 330,295 user-days. A day without an extracted session has no text and is outside this experiment. Days have between one and nine sessions, with two thirds having one. The longest records contain about 1,300 tokens, within the model's input limit. We evaluate each recorded day's predictability from its own preceding tokens; we do not forecast the next day or generate replacement activity.

We split by user so that all days of one employee stay together. Let $\mathcal{U}_{\text{train}}$ denote the training users, $\mathcal{U}_{\text{val}}$ the held-out benign users, and $\mathcal{U}_{\text{ins}}$ the official insiders. The subscripts name the three groups; *val* is the historical name of the held-out group. All 70 official insiders enter the last group. A fixed hash, a repeatable numerical mapping of each remaining user's identifier, assigns about one tenth of those users to the held-out group and the rest to training. QLoRA adaptation makes one pass over the 287,827 days of the 851 training users.

Table 3.2: User-disjoint splits of CERT release 4.2: no user belongs to more than one group.

group	users	user-days	incident-window days
$\mathcal{U}_{\text{train}}$ (training)	851	287,827	0
$\mathcal{U}_{\text{val}}$ (held-out benign)	79	27,026	0
$\mathcal{U}_{\text{ins}}$ (insiders)	70	15,442	1,362

No insider or held-out benign user enters adaptation. Here *held out* means excluded from fine-tuning: these evaluation users have been examined during research development and are not a newly untouched test set.

The evaluation pool contains all 42,468 observed session-days of the 70 insiders and 79 held-out benign users. An *incident-window day* overlaps an insider's recorded incident period under the corrected calendar rule; there are 1,362 such observed days. The insider's other

days are *window-negative*, which does not establish that every activity on them was benign. The label correction changed evaluation labels and score coverage; the adapted model and its benign training membership remain unchanged.

Evaluation asks two distinct questions: do insider users rank above benign users, and do incident-window days rank above negative days? User comparisons use each user's maximum daily score; day comparisons use the individual daily scores. Success on one question does not guarantee success on the other.

A score threshold flags records above it. As that threshold changes, the receiver operating characteristic (ROC) curve plots the fraction of positives flagged against the fraction of negatives flagged. Its area, ROC-AUC or simply AUC, has a useful ranking interpretation: the fraction of positive–negative pairs in which the positive has the higher score, with half credit for ties. An AUC of 1 means perfect ranking; 0.5 is chance-level ranking. The user-level calculation and its connection to fold averaging are written out in Appendix A.5.

For the day-level evaluation, a comparison called a *fold* contains all days of one insider and all days of the same 79 held-out benign users. The insider's incident-window days are positive; their window-negative days and the benign-user days are negative. We calculate the metrics in each of the 70 folds and average them equally. The model stays fixed throughout: these folds do not train 70 separate models. Each user-level fold instead compares that insider's maximum with the 79 benign maxima.

We also report *average precision* (AP). At a threshold, precision is the fraction of flagged cases that are positive, and recall is the fraction of all positive cases that are flagged. AP adds precision values across successive score thresholds, weighting each by the increase in recall. It summarizes a ranking rather than precision at one chosen alert threshold. AP depends on *prevalence*, the fraction of cases that are positive. Mean day-level fold prevalence is about 0.0714% because each fold contains only one insider alongside all 79 benign users. Pooling the 42,468 days instead gives 3.21% positive days. The reported day metrics are fold means, not metrics computed once on that pooled population.

3.6 Results and the Question of Internal Mechanism

Table 3.3 compares three ways to count predictions from the same frozen model and input: all tokens, behavior tokens, and profile tokens. All three use the same corrected population and evaluation procedure. User AUC rises from 0.589 to 0.825 when only behavior-target

Table 3.3: Corrected CERT r4.2 score decomposition: 42,468 days, including 1,362 incident-window days, from 70 insiders and 79 held-out benign users. All views retain the same input and model. AUC and AP columns are equal-insider fold means.

Scored targets	day AUC	day AP	user AUC
All tokens	0.540	0.004859	0.589
Behavior tokens	0.820	0.025238	0.825
Profile tokens	0.482	0.001353	0.476

losses enter the daily average. With half credit for ties, insiders then rank above benign users in about 82.5% of evaluated user pairs, compared with 58.9% for the full score. These are ranking proportions, not percentages of users correctly classified at a particular alert threshold.

To describe uncertainty in the change, we use a *user bootstrap*: repeatedly sample the 70 insiders with replacement and recalculate the mean gain, keeping each insider's full-score and behavior-only AUCs paired. Sampling with replacement allows the same insider to appear more than once in a resampled group. The estimated user-AUC gain is 0.236, with a descriptive 95% user-bootstrap interval $[0.185, 0.289]$; the brackets give its lower and upper endpoints. The model and the benign comparison cohort stay fixed, so this interval does not measure variation across independently trained models.

Behavior-only scoring also improves the displayed day metrics. Their AP values must be read with the small fold prevalence described above, not the larger pooled prevalence. The profile-only user-AUC estimate is slightly below chance, but that point estimate alone does not establish a reliable reversed ranking. The paired comparison directly supports the benefit of changing which predictions are counted in this experiment.

The scope of the conclusion is specific. We changed score composition, not the model, its input, or its learned representations. Profile text can still affect behavior predictions even when its own prediction losses are excluded from the average. The result applies to this model, representation, and synthetic benchmark, whose evaluation users were repeatedly examined during development. Better user ranking also does not establish that a user's highest-scoring day is their incident day.

We have therefore answered a question about the model's output: which predictions should contribute to the anomaly score? The next question concerns the computation producing those predictions. The hidden activations introduced in Section 3.2 provide a place to investigate it. Chapter IV compares those activations before and after adaptation, describes their differences with sparse features, and tests what happens to prediction loss when selected contributions are removed.

Chapter IV

Interpretable Dimensionality Reduction of the Fine-Tuned Model

4.1 Overview and Motivation

Chapter III examined the score from the outside: which tokens its surprisal comes from. *Mechanistic interpretability* asks what the model computes internally [17, 18]. This chapter answers that question with a form of dimensionality reduction designed for interpretation. PCA (Chapter II) reduces data to a few dense directions, each of which mixes many original features. A *sparse autoencoder* instead describes each observation with a few directions drawn from a much larger dictionary; because only a few are active at once, the directions, or *features*, provide candidate patterns to investigate. Sparsity alone does not guarantee that a feature has one interpretable meaning. We apply it to the change that benign fine-tuning made to the model's internal representations, and test two things about the features that best separate malicious from benign days: where they activate, and whether editing them changes the anomaly score.

4.2 Hidden States and the Adaptation Difference

The model processes text through a stack of 36 layers. After each layer, each token t is represented by a *hidden state* $\mathbf{h}_t \in \mathbb{R}^d$ with $d = 4096$. Running the same text through the

model before fine-tuning (the *base* model) and after it (the *adapted* model) gives two hidden states for every token. Their difference,

$$\boldsymbol{\delta}_t \;=\; \mathbf{h}_t^{\text{ada}} - \mathbf{h}_t^{\text{base}} \;\in\; \mathbb{R}^d, \tag{4.1}$$

is the net difference in token t's representation between the two model runs. We study the hidden states after layer 26.

4.3 A Sparse Dictionary and a Worked Example

Recall the PCA reconstruction of Chapter II: a centered observation is projected onto $q < p$ orthonormal directions and reconstructed by their weighted sum. A sparse autoencoder (SAE) also minimizes reconstruction error, but uses an overcomplete dictionary of $m > d$ directions and activates at most k directions for each token [20].

Let $\boldsymbol{\mu}$ and the positive coordinate scales $\boldsymbol{\sigma}$ be estimated from the dictionary's fitting data, and put $S = \operatorname{diag}(\boldsymbol{\sigma})$. The implementation first standardizes each difference vector and then encodes it:

$$\begin{aligned} \widetilde{\boldsymbol{\delta}}_t &= S^{-1}(\boldsymbol{\delta}_t - \boldsymbol{\mu}), \\ \mathbf{z}_t &= \operatorname{TopK}_k\Big(\operatorname{ReLU}(W_{\text{enc}}\widetilde{\boldsymbol{\delta}}_t + \mathbf{b}_{\text{enc}})\Big). \end{aligned} \tag{4.2}$$

ReLU sets negative encoder outputs to zero, and TopK retains the k largest entries. Some of those entries may already be zero, so the code has at most k nonzero values. The reconstruction in the model's original coordinates is

$$\widehat{\boldsymbol{\delta}}_t = \boldsymbol{\mu} + SW_{\text{dec}}\mathbf{z}_t = \boldsymbol{\mu} + S\sum_f z_{f,t}\mathbf{w}_f, \tag{4.3}$$

where $\mathbf{w}_f$ is decoder column f. Training minimizes squared reconstruction error in standardized coordinates. The decoder has no additional learned bias.

Here $d = 4096$, $m = 8192$ and $k = 4$. The corrected dictionary is fitted for 20 epochs on two million sampled token differences at layer 26. The reconstruction objective has no label term, but the sampling procedure is label-aware: it preserves the 171,917 token rows from discovery-positive days and samples the remaining rows from the rest of the evaluation population. Confirmation-user inputs can therefore enter dictionary fitting, although their labels do not prioritize that sampling or enter feature ranking. This is a *transductive* dictionary, not a dictionary fitted on an independent training population. The low-dimensional object is each token's sparse code; the dictionary as a whole has more directions than the original space.

For an illustration, take $d = 2$, a dictionary of $m = 3$ unit directions $\mathbf{w}_1 = (1,\ 0)$, $\mathbf{w}_2 = (0,\ 1)$ and $\mathbf{w}_3 = (0.6,\ 0.8)$, one active feature ($k = 1$), $\boldsymbol{\mu} = \mathbf{b}_{\text{enc}} = \mathbf{0}$, $S = I$, and $W_{\text{enc}} = W_{\text{dec}}^{\top}$. For $\boldsymbol{\delta} = (1.2,\ 1.6)$ the scores are $\mathbf{a} = (1.2,\ 1.6,\ 2.0)$, so feature 3 is kept and $\widehat{\boldsymbol{\delta}} = 2.0\,\mathbf{w}_3 = (1.2,\ 1.6)$ with zero error. With only the two axes available, the best single direction would leave an error of $1.2^2 = 1.44$. A different token, $\boldsymbol{\delta}' = (1.0,\ 0.1)$, is described by feature 1 with error 0.01: each token uses the few features that fit it best, which is *sparse coding*.

4.4 Choosing the Features to Test

The 70 insiders are divided into 35 discovery and 35 confirmation users. Feature selection uses corrected discovery labels and excludes the confirmation users. For each feature f, the ranking statistic is

$$g_f = \bar{z}_f^{(+)} - \bar{z}_f^{(-)}, \tag{4.4}$$

the difference between its mean token activation in the positive and negative discovery populations. The selected set $\mathcal{F}$ contains the five largest gaps. The active control set $\mathcal{C}$ uses five other features with gaps near zero and nonzero activation on at least 0.2% of ranking tokens. This activity threshold does not match either the activation frequency or the physical size

of the selected edit.

The corrected selected IDs are 3843, 3230, 3199, 3975 and 1394. IDs identify columns of this particular fitted dictionary; they are not named concepts and cannot be compared directly with IDs from another fit. The confirmation group excludes users from feature ranking, but those users were inspected earlier in the research and enter unlabelled dictionary fitting. It is therefore a check across ranking groups, not an independent confirmation set.

4.5 Where Features Activate

For a feature f and a token class c, the share of the feature's total activation that lands on class-c tokens is

$$\rho_f(c) \;=\; \frac{\sum_{t\in\mathcal{T}_c} z_{f,t}}{\sum_{t\in\mathcal{T}} z_{f,t}}, \tag{4.5}$$

and its *enrichment* compares that share with the class's share of the tokens,

$$E_f(c) \;=\; \frac{\rho_f(c)}{N_c/N}.$$

When the denominator is positive, an enrichment of 1 means activation mass is proportional to token frequency; it does not measure how often the feature fires. Larger values mean more activation mass per token on class c, up to the maximum $1/(N_c/N)$ reached when all of a feature's activation lands on that class.

If a feature has zero total activation in a group, its location fraction is undefined. Table 4.1 reports session-token mass and the number of confirmation users on whose incident-window days the feature activates at least once. Four features place virtually all of their incident-window activation mass on session tokens. Feature 3843 has a mixed location and much weaker confirmation activity. Location answers where a feature activates. It does not identify all information carried there: a session state can depend on the profile text preceding it. Nor does location alone show which predictions would change if that feature were edited.

Table 4.1: Corrected r4.2 attribution. The discovery group has 695 incident-window days from 35 users; confirmation has 667 days from 35 users. Percentages pool activation mass within each group. Values displayed as 100.00% are rounded.

Feature	discovery SES (%)	confirmation SES (%)	active users
3843	91.45	84.95	13/35
3230	100.00	100.00	32/35
3199	100.00	100.00	24/35
3975	100.00	100.00	30/35
1394	100.00	100.00	30/35

4.6 Causal Test: Removing Features and Re-Scoring

Attribution is descriptive. To test whether the selected representation contributes to prediction, remove its decoded contribution from the adapted hidden state:

$$\mathbf{h}'_t = \mathbf{h}^{\text{ada}}_t - \alpha S \sum_{f \in \mathcal{F}} z_{f,t} \mathbf{w}_f, \qquad 0 \le \alpha \le 1. \tag{4.6}$$

This leaves the original hidden state and its reconstruction residual intact except for the specified subtraction. At $\alpha = 0$ it is an exact no-op; at $\alpha = 1$ it removes the selected decoded contribution. Tokens with no selected activation remain unchanged. Re-encoding the edited state need not reproduce the intended sparse code, so the experiment is defined by this physical subtraction, not by assuming independent feature coordinates.

The remaining model layers predict the same target tokens. For a day D, let $\Delta_{\mathcal{F}}(D) = \bar{\ell}'_B(D) - \bar{\ell}_B(D)$ be the change in behavior-target loss. A positive value means prediction became worse. Two controls answer different questions: removal of the active-control features tests a separate feature-selection policy, while a random dictionary direction at the same edited positions is matched to the realized per-token edit norm within 5%. Matching is checked after numerical rounding in the model's precision. The fixed feasibility policy for those dictionary controls is retained from the corrected recovery protocol; it is not a new, independently preregistered test. The active-feature control is not dose matched.

The primary removal strength is fixed at $\alpha = 1$. For each incident-window receiver, the experiment also uses a same-user window-negative day, chosen by nearest calendar day, preferring the same weekday and using fixed hash tie-breaking. Negative days can be reused. For a receiver pair (D^+, D^-), the relative window effect is

$$\begin{aligned}\tau(D^+, D^-) = [\Delta_{\mathcal{F}}(D^+) - \Delta_{\mathcal{C}}(D^+)] \\ - [\Delta_{\mathcal{F}}(D^-) - \Delta_{\mathcal{C}}(D^-)].\end{aligned} \tag{4.7}$$

A positive value would indicate a larger selected-versus-active response inside the incident window. Pairing is observational: even a positive value would not by itself prove an attack-specific circuit. Days are averaged within each user, then users are weighted equally; zero edits and repeated negative-day appearances retain their prescribed weights.

4.7 Corrected Mechanistic Results

The corrected run covers all 1,362 incident-window days and 380 unique matched window-negative days, giving 1,742 receivers and 13,936 scored rows across eight conditions. The primary summary uses the 667 incident-window days of the 35 confirmation users. Zero-edit and implementation checks passed, and the aggregate estimates were independently reproduced.

Table 4.2: Behavior-target loss changes at full removal strength on the 35 confirmation users, in nats per token. Intervals are descriptive, unadjusted 95% intervals from 10,000 user-bootstrap draws. The final row is the incident-minus-window-negative contrast in (4.7); the other rows describe incident-window receivers.

Contrast	mean	95% interval
Selected minus baseline	0.004411	[0.001812, 0.007283]
Selected minus matched direction	0.005531	[0.002886, 0.008625]
Selected minus active control	0.004275	[0.001709, 0.007179]
Incident minus window-negative	−0.004599	[−0.009430, −0.000616]

Removing the selected contribution raises behavior loss relative to baseline and to both

specified controls. This supports a contribution to prediction under this edit. However, the relative response is smaller on incident-window days than on their matched window-negative days: the final contrast is negative. The results therefore do not support an enhanced attack-specific response under this policy.

An intervention's effect on prediction loss is also different from an improvement in anomaly detection. These estimates do not demonstrate better ranking, semantic identification of the features, or mediation of the profile's influence. Together with Chapter III, they support two bounded conclusions: score composition affects insider ranking, and the selected sparse directions contribute to behavior prediction. The second does not explain the first by itself.

4.8 Limitations and Conclusions

The analysis uses one adapted model and one synthetic benchmark release. Model-seed variation and transfer to real organizational data remain untested here. The layer and dictionary settings were informed by earlier experiments; the corrected dictionary is transductive, its sample prioritizes discovery-positive rows, and its confirmation users were previously inspected. Feature ranking excludes those users, but that separation does not make the entire analysis independent.

The controls have distinct scopes. Active-feature removal compares selection policies with different natural edit sizes. Dictionary directions match edited positions and realized norms, but do not enumerate every possible alternative intervention. Same-user negative days control some user differences but are not verified benign ground truth. The reported intervals are conditional on the fixed model, dictionary, feature sets, pairs and controls.

Finally, an activation at a behavior-token position need not represent behavior alone. Interpretable labels for individual features require further evidence, and these joint edits do not identify a complete computational circuit. The corrected ablation is the causal test

reported in this thesis; the earlier donor-search result is not carried forward as current evidence after label correction.

Appendix A

Supporting Mathematics for Language-Model Scoring

This appendix supplies the derivations supporting Chapter III. The main chapter develops the prediction task and its scoring rule; the arguments below explain why the logarithm is useful, how it relates to the output scores, and how the evaluation averages are constructed. All logarithms here are natural logarithms.

A.1 Why Additive Surprise Has a Logarithmic Form

Let p be an event's probability, with $0 < p \leq 1$, and let $S(p)$ be the amount of surprise assigned to it. Require that S strictly decreases as p increases: rarer events are more surprising. Let q be the probability of a second, independent event, with $0 < q \leq 1$. Independence makes the probability of both events pq. Requiring their surprises to add gives

$$S(pq) = S(p) + S(q). \tag{A.1}$$

This is a requirement on the surprise measure for independent events; it does not assert that successive text tokens are independent.

To find its form, introduce nonnegative real numbers $x = -\ln p$ and $y = -\ln q$, so $p = e^{-x}$ and $q = e^{-y}$. Here e is the base of the natural exponential. Define a new function $g(x) = S(e^{-x})$. Equation (A.1) becomes

$$g(x + y) = g(x) + g(y).$$

The function g is increasing because e^{-x} decreases with x and S decreases with probability. This additive relation is Cauchy's functional equation. Its increasing solutions on nonnegative real numbers have the form $g(x) = cx$, where c is a positive constant. Substitution therefore gives

$$S(p) = -c \ln p, \qquad c > 0. \tag{A.2}$$

The constant c sets the measurement unit. Choosing $c = 1$ gives surprisal in nats, as used in Chapter III [19]. A certain event, with $p = 1$, has zero surprise. As p approaches zero, surprise grows without bound. Every tenfold decrease in probability adds $\ln 10$ nats. Applying the logarithm to the chain of conditional token probabilities gives (3.6) without requiring independent tokens.

A.2 Surprisal Written in Terms of Logits

Fix one prediction position t, where x_t is the observed token and $x_{<t}$ is the preceding sequence. Let θ denote the fixed model parameters and $\mathcal{V}$ its vocabulary. Its size is $V = |\mathcal{V}|$, where the bars count members of the set. For each candidate token v in the vocabulary, let o_v be its output logit, and let $\mathbf{o}$ collect all those logits. Define the *log-sum-exp* function, abbreviated LSE, by taking the logarithm of the sum of exponentiated logits. Substituting softmax into the observed token's surprisal ℓ_t gives

$$\begin{aligned} \ell_t &= -\log p_\theta(x_t \mid x_{<t}) = \mathrm{LSE}(\mathbf{o}) - o_{x_t}, \\ \mathrm{LSE}(\mathbf{o}) &= \log \sum_{v \in \mathcal{V}} e^{o_v}. \end{aligned} \tag{A.3}$$

The subscript x_t selects the logit of the observed token. The summation adds over all vocabulary members, and e is the natural exponential base.

Let $o_{\max} = \max_{v \in \mathcal{V}} o_v$ be the largest logit. The sum of exponentials includes at least one

term $e^{o_{\max}}$, and none of its V terms is larger than that term. Thus

$$e^{o_{\max}} \leq \sum_{v \in \mathcal{V}} e^{o_v} \leq V e^{o_{\max}}.$$

Taking logarithms, which preserve these inequalities, gives

$$o_{\max} \leq \mathrm{LSE}(\mathbf{o}) \leq o_{\max} + \log V.$$

For two candidate tokens a and b, the common softmax denominator cancels in their probability ratio:

$$\frac{p_\theta(x_t = a \mid x_{<t})}{p_\theta(x_t = b \mid x_{<t})} = e^{o_a - o_b}.$$

Only logit differences determine the probabilities. Having the largest logit alone does not guarantee a small surprisal: many other candidates may have similar logits and share the probability.

A.3 Maximum Likelihood in a Frequency Model

Consider a model that assigns a fixed probability to each of K token types, regardless of preceding text. Let j index those types from 1 to K, let N_j count occurrences of type j in the training observations, and let $N = \sum_{j=1}^{K} N_j$ be the total count. Write p_j for the probability assigned to type j and $\mathbf{p} = (p_1, \ldots, p_K)$ for their collection. For the interior calculation, suppose each N_j is positive and each p_j is positive.

The summed negative log-likelihood is $-\sum_{j=1}^{K} N_j \log p_j$, subject to the probabilities adding to one. Introduce a real number λ, the Lagrange multiplier, which enforces this constraint. Define the Lagrangian $\mathcal{L}(\mathbf{p}, \lambda)$ to be the objective plus λ times the constraint residual:

$$\mathcal{L}(\mathbf{p}, \lambda) = -\sum_{j=1}^{K} N_j \log p_j + \lambda \left(\sum_{j=1}^{K} p_j - 1 \right).$$

The parenthesized expression is zero exactly when the probabilities sum to one. Differentiating with respect to each p_j while holding the other quantities fixed, and setting the derivative to zero, gives

$$\frac{\partial \mathcal{L}}{\partial p_j} = -\frac{N_j}{p_j} + \lambda = 0, \qquad p_j = \frac{N_j}{\lambda}.$$

The symbol ∂ denotes this partial derivative. Summing the last equality over token types and imposing a total probability of one gives $\lambda = N$. Hence the minimizing probabilities are

$$\widehat{p}_j = \frac{N_j}{N}, \qquad j = 1, \ldots, K. \tag{A.4}$$

The hat marks a fitted value. The objective is convex: along a straight segment between two positive probability vectors, its value lies at or below the line joining the endpoint values. Thus this stationary point gives the minimum under the constraint. If additional types have zero observed count, the optimum in the unrestricted frequency model assigns them zero probability. This frequency calculation illustrates the objective; a contextual language model with shared parameters and finite softmax logits is a different model and need not reproduce these exact frequencies.

A.4 Why Subtracting Class Means Gives a Different Score

For one day, let N_P and N_B count scored profile and behavior tokens, and let $N = N_P + N_B$ be their total. Assume both classes are nonempty. Let $\bar{\ell}_P$ and $\bar{\ell}_B$ be their average surprisals, and let $\bar{\ell}$ be the average across all scored tokens. From (3.11),

$$\begin{aligned} \bar{\ell} - \bar{\ell}_P &= \frac{N_P}{N}\bar{\ell}_P + \frac{N_B}{N}\bar{\ell}_B - \bar{\ell}_P \\ &= \frac{N_B}{N}\big(\bar{\ell}_B - \bar{\ell}_P\big). \end{aligned} \tag{A.5}$$

The result is a scaled difference between class means, not the behavior mean. For a numerical illustration, take two days, named C and D, each with behavior-token share $N_B/N = 0.8$. Give Day C profile mean 3.0 and behavior mean 1.2, and Day D profile mean 0.2 and behavior mean 0.8. Subtracting means gives $0.8(1.2 - 3.0) = -1.44$ for C and $0.8(0.8 - 0.2) = 0.48$ for D. It ranks D above C even though C's behavior has higher surprisal. Removing a token class from an average requires removing both its summed loss and its token count, as in (3.12).

A.5 User AUC and Equal-Insider Fold Averages

Fix one scoring rule. Let M be the number of insider users and n_- the number of benign comparison users, both positive. The minus subscript labels the negative class; it does not mean subtraction. Let m index insiders from 1 to M and j index benign users from 1 to n_-. Write s_m^+ for insider m's maximum daily score and s_j^- for benign user j's maximum daily score. The superscripts + and − label positive and negative classes, not powers.

Define the indicator $\mathbb{I}[\text{condition}]$ to be 1 when the enclosed condition is true and 0 otherwise. The user AUC for the fold containing insider m is

$$\text{AUC}_m = \frac{1}{n_-} \sum_{j=1}^{n_-} \left(\mathbb{I}[s_m^+ > s_j^-] + \tfrac{1}{2} \mathbb{I}[s_m^+ = s_j^-] \right). \tag{A.6}$$

Each comparison contributes 1 if the insider outranks the benign user, one half for a tie, and 0 otherwise. Let a_{mj} name that contribution for pair (m, j). The overall equal-insider mean, denoted AUC, therefore satisfies

$$\text{AUC} = \frac{1}{M} \sum_{m=1}^{M} \text{AUC}_m = \frac{1}{M} \sum_{m=1}^{M} \frac{1}{n_-} \sum_{j=1}^{n_-} a_{mj} = \frac{1}{Mn_-} \sum_{m=1}^{M} \sum_{j=1}^{n_-} a_{mj}.$$

Every insider is compared with the same benign users, so this is also the pooled user AUC over all Mn_- pairs. That identity does not establish an analogous equality for day-level fold

averages or AP, whose comparison populations and weights differ.

For illustration, let four benign users have full scores $(0.93, 0.70, 0.85, 0.60)$ and one insider have score 0.77. The insider outranks two benign users, giving AUC $2/4 = 0.50$. If the behavior-only scores of those users are $(0.41, 0.55, 0.70, 0.48)$ and the insider's is 0.93, the insider outranks all four, giving AUC $4/4 = 1.00$. These examples explain the calculation and are not reported experimental observations.

REFERENCES

Bibliography

[1] Pearson, K. (1901). On lines and planes of closest fit to systems of points in space. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 2(11), 559–572.

[2] Hotelling, H. (1933). Analysis of a complex of statistical variables into principal components. *Journal of Educational Psychology*, 24(6), 417–441.

[3] Jolliffe, I. T. (1986). *Principal Component Analysis*. Springer Series in Statistics. Springer, New York. https://doi.org/10.1007/978-1-4757-1904-8

[4] Software Engineering Institute. (2016). *Insider Threat Test Dataset*. Carnegie Mellon University Software Engineering Institute. https://doi.org/10.1184/R1/12841247.v1

[5] Glasser, J., & Lindauer, B. (2013). Bridging the gap: A pragmatic approach to generating insider threat data. In *2013 IEEE Security and Privacy Workshops* (pp. 98–104). IEEE. https://doi.org/10.1109/SPW.2013.37

[6] Le, D. C., Zincir-Heywood, N., & Heywood, M. I. (2020). Analyzing data granularity levels for insider threat detection using machine learning. *IEEE Transactions on Network and Service Management*, 17(1), 30–44. https://doi.org/10.1109/TNSM.2020.2967721

[7] Shlens, J. (2014). *A Tutorial on Principal Component Analysis*. arXiv. https://doi.org/10.48550/arXiv.1404.1100

[8] Cappelli, D., Moore, A., & Trzeciak, R. (2012). *The CERT Guide to Insider Threats: How to Prevent, Detect, and Respond to Information Technology Crimes (Theft, Sabotage, Fraud)*. Addison-Wesley Professional.

[9] HPC Wiki, The University of Southern Mississippi. (2024). *Main Page*. Magnolia HPC Wiki. Suggested infrastructure acknowledgment: The authors acknowledge HPC at The University of Southern Mississippi supported by the National Science Foundation under the Major Research Instrumentation (MRI) program via Grant # ACI 1626217. `http://magnolia.usm.edu/index.php?title=Main_Page&oldid=1116`

[10] Du, M., Li, F., Zheng, G., & Srikumar, V. (2017). DeepLog: Anomaly detection and diagnosis from system logs through deep learning. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security*.

[11] Tuor, A., Kaplan, S., Hutchinson, B., Nichols, N., & Robinson, S. (2017). Deep learning for unsupervised insider threat detection in structured cybersecurity data streams. In *AAAI Workshop on Artificial Intelligence for Cyber Security*.

[12] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems*.

[13] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2022). LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*.

[14] Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems*.

[15] Qwen Team. (2025). *Qwen3 technical report*. arXiv preprint arXiv:2505.09388.

[16] Qwen Team. (2025). *Qwen3-8B model configuration*. https://huggingface.co/Qwen/Qwen3-8B/blob/main/config.json. Accessed October 14, 2026.

[17] Bricken, T., Templeton, A., Batson, J., Chen, B., Jermyn, A., Conerly, T., Turner, N. L., et al. (2023). Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*. https://transformer-circuits.pub/2023/monosemantic-features

[18] Cunningham, H., Ewart, A., Riggs, L., Huben, R., & Sharkey, L. (2023). Sparse autoencoders find highly interpretable features in language models. arXiv preprint arXiv:2309.08600.

[19] Shannon, C. E. (1948). A mathematical theory of communication. *The Bell System Technical Journal*, 27(3), 379–423.

[20] Gao, L., Dupré la Tour, T., Tillman, H., Goh, G., Troll, R., Radford, A., Sutskever, I., Leike, J., & Wu, J. (2024). Scaling and evaluating sparse autoencoders. arXiv preprint arXiv:2406.04093.